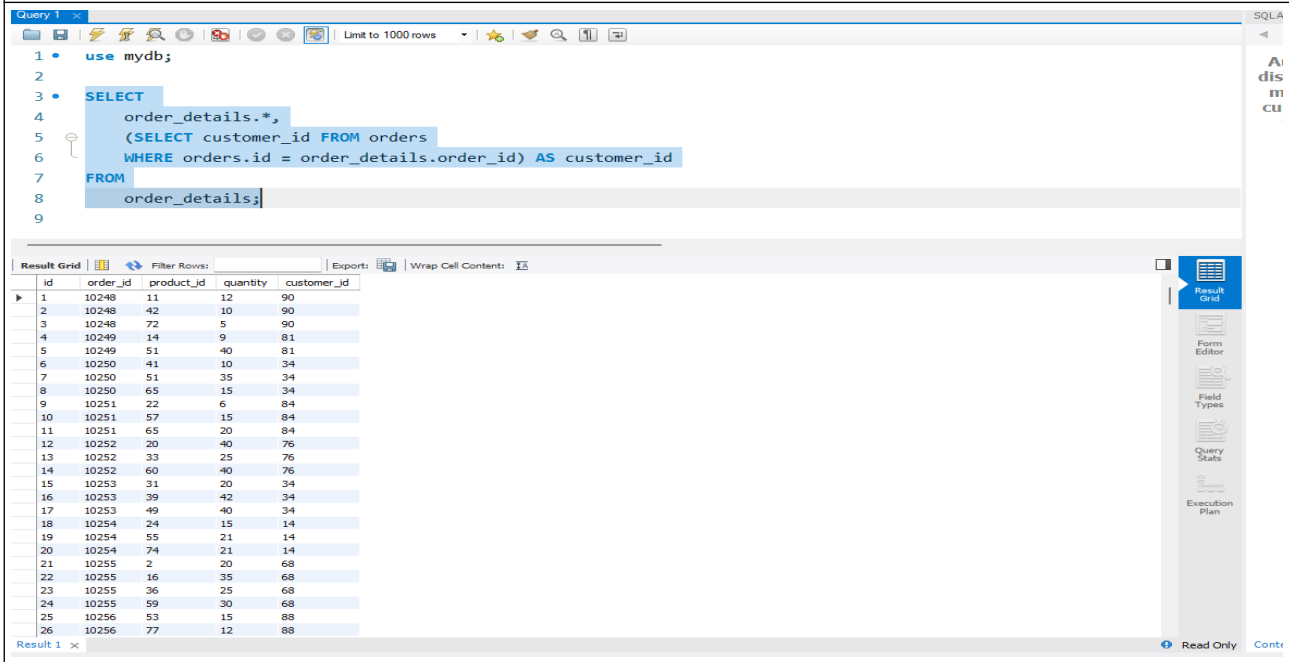
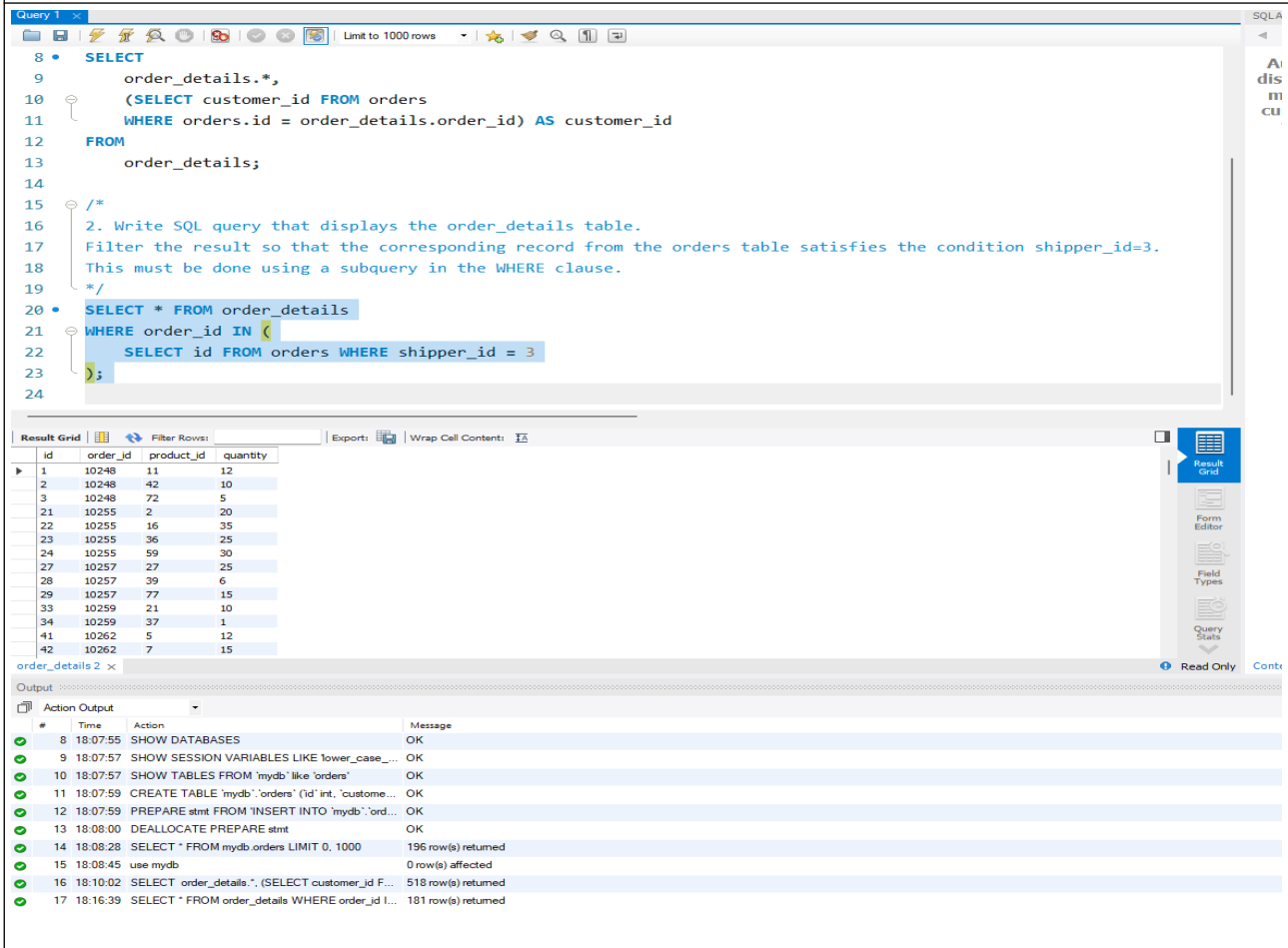
1. Write SQL query that displays the order_details table and the custom_id field from the orders table corresponding to each record on the order_details table. This must be done using subquery in SELECT statement.



```
use mydb;

SELECT
    order_details.*,
    (SELECT customer_id FROM orders
    WHERE orders.id = order_details.order_id) AS customer_id
FROM
    order_details;
```

id	order_id	product_id	quantity	customer_id
1	10248	11	12	90
2	10248	42	10	90
3	10248	72	5	90
4	10249	14	9	81
5	10249	51	40	81
6	10250	41	10	34
7	10250	51	35	34
8	10250	65	15	34
9	10251	22	6	84
10	10251	57	15	84
11	10251	65	20	84
12	10252	20	40	76
13	10252	33	25	76
14	10252	60	40	76
15	10253	31	20	34
16	10253	39	42	34
17	10253	49	40	34
18	10254	24	15	14
19	10254	55	21	14
20	10254	74	21	14
21	10255	2	20	68
22	10255	16	35	68
23	10255	36	25	68
24	10255	59	30	68
25	10256	53	15	88
26	10256	77	12	88

2. Write SQL query that displays the order_details table. Filter the result so that the corresponding record from the orders table satisfies the condition shipper_id=3. This must be done using a subquery in the WHERE clause.



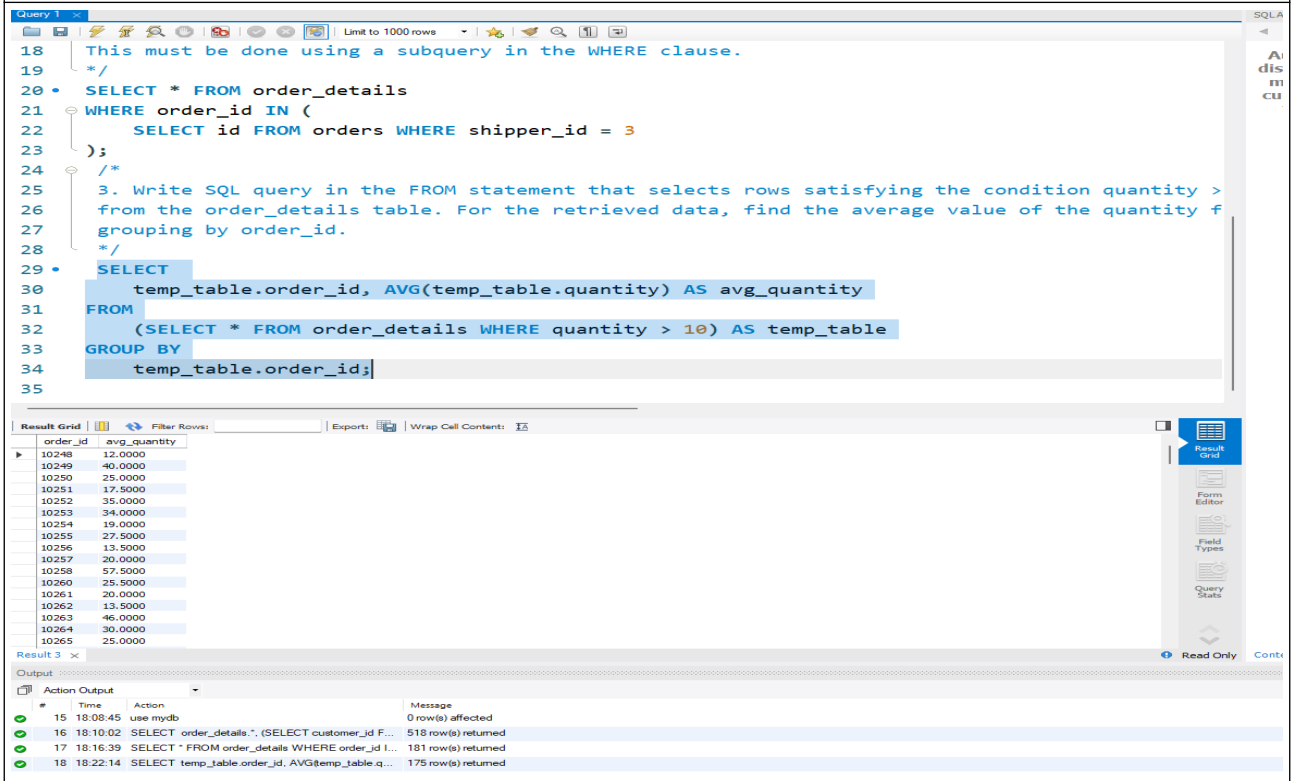
```
SELECT
    order_details.*,
    (SELECT customer_id FROM orders
    WHERE orders.id = order_details.order_id) AS customer_id
FROM
    order_details;

/*
2. Write SQL query that displays the order_details table.
Filter the result so that the corresponding record from the orders table satisfies the condition shipper_id=3.
This must be done using a subquery in the WHERE clause.
*/
SELECT * FROM order_details
WHERE order_id IN (
    SELECT id FROM orders WHERE shipper_id = 3
);
```

id	order_id	product_id	quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
21	10255	2	20
22	10255	16	35
23	10255	36	25
24	10255	59	30
27	10257	27	25
28	10257	39	6
29	10257	77	15
33	10259	21	10
34	10259	37	1
41	10262	5	12
42	10262	7	15

Output

#	Time	Action	Message
8	18:07:55	SHOW DATABASES	OK
9	18:07:57	SHOW SESSION VARIABLES LIKE 'lower_case_...	OK
10	18:07:57	SHOW TABLES FROM `mydb` like 'orders'	OK
11	18:07:59	CREATE TABLE `mydb`.`orders` (`id` int, `custome...	OK
12	18:07:59	PREPARE stmt FROM 'INSERT INTO `mydb`.`ord...	OK
13	18:08:00	DEALLOCATE PREPARE stmt	OK
14	18:08:28	SELECT * FROM mydb.orders LIMIT 0, 1000	196 row(s) returned
15	18:08:45	use mydb	0 row(s) affected
16	18:10:02	SELECT order_details.*, (SELECT customer_id F...	518 row(s) returned
17	18:16:39	SELECT * FROM order_details WHERE order_id I...	181 row(s) returned

3. Write SQL query in the FROM statement that selects rows satisfying the condition quantity > 10 from the order_details table. For the retrieved data, find the average value of the quantity field, grouping by order_id.

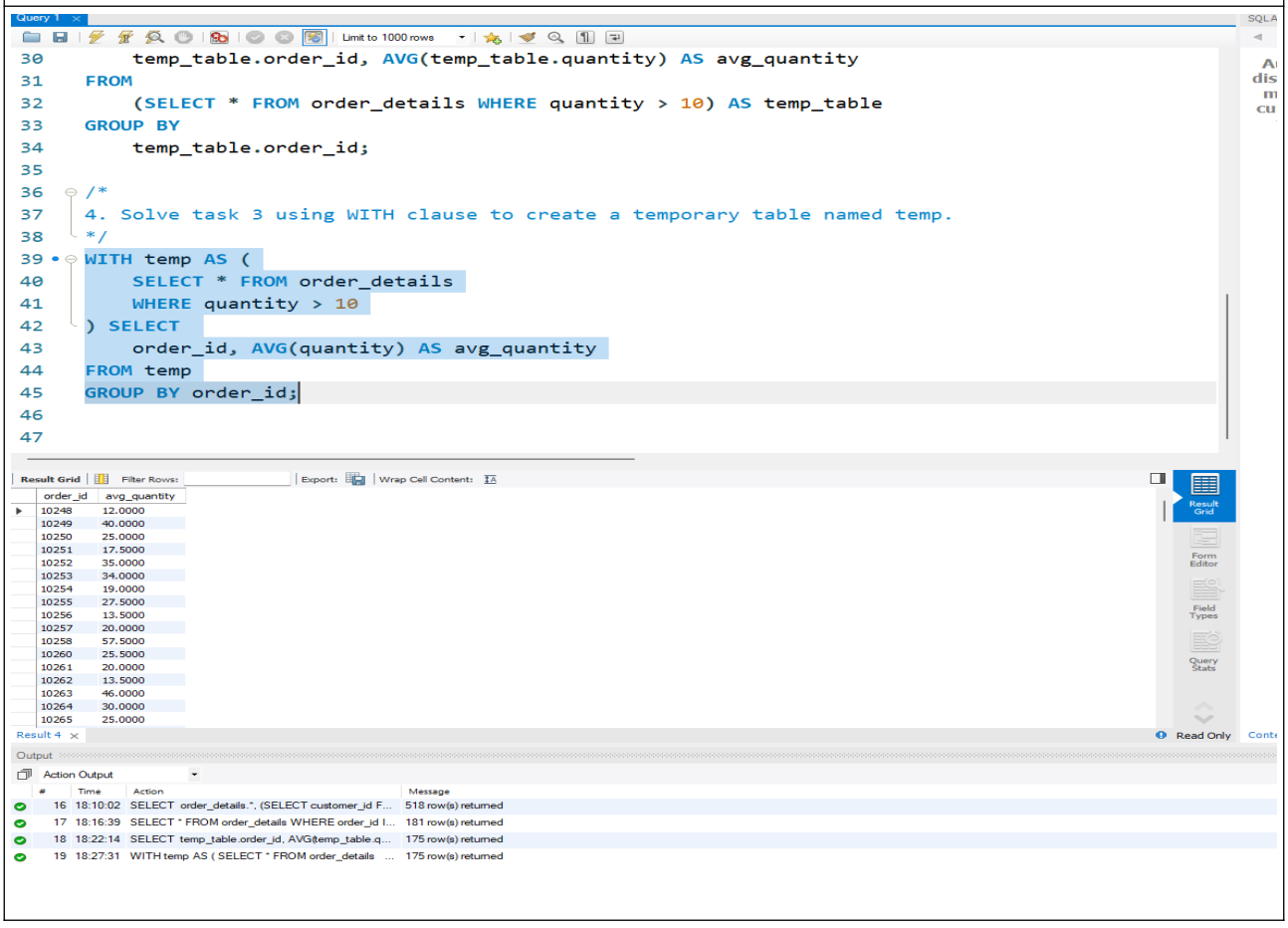


```
This must be done using a subquery in the WHERE clause.
*/
SELECT * FROM order_details
WHERE order_id IN (
    SELECT id FROM orders WHERE shipper_id = 3
);
 /*
 3. Write SQL query in the FROM statement that selects rows satisfying the condition quantity >
 from the order_details table. For the retrieved data, find the average value of the quantity f
 grouping by order_id.
 */
 SELECT
    temp_table.order_id, AVG(temp_table.quantity) AS avg_quantity
FROM
    (SELECT * FROM order_details WHERE quantity > 10) AS temp_table
GROUP BY
    temp_table.order_id;
```

order_id	avg_quantity
10248	12.0000
10249	40.0000
10250	25.0000
10251	17.5000
10252	35.0000
10253	34.0000
10254	19.0000
10255	27.5000
10256	13.5000
10257	20.0000
10258	57.5000
10260	25.5000
10261	20.0000
10262	13.5000
10263	46.0000
10264	30.0000
10265	25.0000

#	Time	Action	Message
15	18:08:45	use mydb	0 row(s) affected
16	18:10:02	SELECT order_details.*, (SELECT customer_id F...	518 row(s) returned
17	18:16:39	SELECT * FROM order_details WHERE order_id I...	181 row(s) returned
18	18:22:14	SELECT temp_table.order_id, AVG(temp_table.q...	175 row(s) returned

4. Solve task 3 using WITH clause to create a temporary table named temp.



```
    temp_table.order_id, AVG(temp_table.quantity) AS avg_quantity
FROM
    (SELECT * FROM order_details WHERE quantity > 10) AS temp_table
GROUP BY
    temp_table.order_id;

/*
4. Solve task 3 using WITH clause to create a temporary table named temp.
*/
WITH temp AS (
    SELECT * FROM order_details
    WHERE quantity > 10
) SELECT
    order_id, AVG(quantity) AS avg_quantity
FROM temp
GROUP BY order_id;
```

order_id	avg_quantity
10248	12.0000
10249	40.0000
10250	25.0000
10251	17.5000
10252	35.0000
10253	34.0000
10254	19.0000
10255	27.5000
10256	13.5000
10257	20.0000
10258	57.5000
10260	25.5000
10261	20.0000
10262	13.5000
10263	46.0000
10264	30.0000
10265	25.0000

#	Time	Action	Message
16	18:10:02	SELECT order_details.*, (SELECT customer_id F...	518 row(s) returned
17	18:16:39	SELECT * FROM order_details WHERE order_id I...	181 row(s) returned
18	18:22:14	SELECT temp_table.order_id, AVG(temp_table.q...	175 row(s) returned
19	18:27:31	WITH temp AS (SELECT * FROM order_details ...	175 row(s) returned

5. Create a function with two parameters that divides the firs parameter by the second. Both parameters and the return value must be type FLOAT. Use the DROP FUNCTION IF EXISTS construct. Apply the function to the quantity attribute of the order_details table. The second parameter can be an attribute number of your choice.